TRADUZIONE DELL’INTRODUZIONE E PRIMO CAPITOLO

Why study Computational Thinking?

I computer sono ovunque nel mondo moderno. Aiutano a gestire quasi ogni aspetto della nostra vita e della nostra società.

Alcuni dei modi in cui ci aiutano sono ovvi. Controlliamo le nostre finanze usando i computer. Li usiamo per gestire le nostre interazioni sociali, ricevere le ultime notizie o organizzare viaggi. Hanno abilitato nuove modalità di partecipazione politica ed è impossibile concepire la moderna vita d'ufficio senza computer.

Tuttavia, molte altre parti della società moderna dipendono dal calcolo in modi che potresti non aver mai considerato. Le tue finanze personali, per quanto sane possano essere, sono solo una goccia nell'oceano rispetto ai miliardi elaborati quotidianamente dai computer in banche e borse valori di tutto il mondo. I contenuti che consumi online ti vengono forniti da enormi data center e vengono creati utilizzando software come elaboratori di testi, editor grafici e database. I computer alimentano le nostre reti di comunicazione globale. Macchinari agricoli computerizzati producono il cibo che mangi. I tuoi beni di consumo vengono assemblati in fabbriche automatizzate. Miliardi di testi e messaggi online vengono scambiati in tutto il mondo tramite software ogni giorno. Chiaramente, i computer sono profondamente radicati nella società e noi umani dipendiamo fortemente da essi.

Eppure, c'è un rovescio della medaglia della rivoluzione informatica che ci espone a numerosi rischi. La criminalità informatica aumenta man mano che sempre più parti della nostra vita vengono vissute online. Le questioni relative alla privacy continuano a essere oggetto di dibattito. Sempre più spesso sono gli algoritmi, non gli umani, a decidere cose come quali notizie leggi e quale potenziale pericolo rappresenti. "Il computer dice di no" diventa meno una battuta e più una frase sinistra dal nostro futuro controllato dai computer.

Altrettanto chiaramente, quindi, la società ha il dovere di preparare tutti a vivere in un mondo del genere. I computer sono i nostri strumenti. Dovrebbero essere al nostro servizio e non il contrario. Dovrebbero potenziarci, ispirarci ed essere una forza benefica nella nostra vita. Per contribuire a garantire ciò, è importante capire come funzionano e cosa sono in grado di fare. A tal fine, il pensiero computazionale (spesso abbreviato in CT) distilla importanti lezioni e principi dall'informatica (CS), l'area tematica che ci insegna come piegare quelle macchine alla nostra volontà. Queste lezioni e principi includono come individuare i dettagli essenziali di un problema, come formulare un problema in modi comprensibili a un computer e come seguire un processo di risoluzione dei problemi in modi che il processo possa essere automatizzato. Questi dovrebbero essere importanti per chiunque, indipendentemente dal fatto che lavori o meno nell'informatica, perché lo scopo ultimo della loro distillazione (al di là della semplice comprensione) è di consentire a tutti di essere in grado non solo di risolvere un problema, ma anche di incorporare un computer in una soluzione per svolgere il compito più rapidamente ed efficiente.

Se non sai di cosa è capace un computer, non puoi certo farlo fare cose per te.

Per coloro che intendono intraprendere una carriera nello sviluppo di software, ci sono ovvi vantaggi nell'apprendere il CT. Collega un metodo di analisi dei problemi con la conoscenza e la tecnologia dell'CS, fornendo competenze fondamentali per la risoluzione dei problemi rilevanti per

la produzione di soluzioni di alta qualità. Studiando CT, imparerai i modi per rendere il software che sviluppi più robusto, potente, ben progettato, ampiamente applicabile e privo di errori.

Ma anche se non sei intenzionato a una carriera di programmatore, il CT ha comunque importanti lezioni da imparare. Potresti non diventare un programmatore professionista, ma probabilmente incontrerai la programmazione in qualche modo. Numerosi documenti di ricerca discutono l'importanza del pensiero computazionale in molte diverse carriere, come:

- scienze naturali:
    - biologia computazionale;
    - genomica;
    - fisica applicata;
    - cambiamento climatico;
    - astronomia.
- scienze sociali:
    - studi sociali;
    - analisi della popolazione.
- medicina:
    - analisi delle malattie;
    - imaging medico;
    - pratica clinica.
- linguistica;
- legge;
- musica;
- insegnamento.

"È quasi impossibile fare ricerca in qualsiasi disciplina scientifica o ingegneristica senza la capacità di pensare computazionalmente. (Carnegie Mellon, 2016)

Infine, oltre ad essere importante in una vasta gamma di campi, il CT è persino promosso come un'abilità di vita. Jeannette Wing, che ha fatto molto per promuovere l'idea del pensiero computazionale sin dal suo storico discorso nel 2006 (Wing, 2006), sottolinea l'importanza del CT per la vita di tutti i giorni e suggerisce che tutti possono trarre qualcosa dallo studio di esso:

Ho sostenuto che l'uso di concetti, metodi e strumenti computazionali avrebbe trasformato la stessa condotta di ogni disciplina, professione e settore. Qualcuno con la capacità di usare il calcolo in modo efficace avrebbe un vantaggio su qualcuno senza. Quindi, ho visto una grande opportunità per la comunità dell'informatica di insegnare alle future generazioni come pensano gli informatici. Quindi "pensiero computazionale"...

Il pensiero computazionale sarà un'abilità fondamentale utilizzata da tutti nel mondo entro la metà del 21° secolo. Per fondamentale, intendo fondamentale come leggere, scrivere e fare aritmetica. (Wing, 2014)

Questo libro è una guida al tema del pensiero computazionale e di come può aiutarti nell'apprendimento dello sviluppo software. Ti guiderà attraverso tutti i concetti fondamentali del tema, spiegando ciascuno con definizioni, discussioni ed esempi. Il materiale è integrato da numerose regole, suggerimenti, consigli e riferimenti per ulteriori letture. E avrai l'opportunità di mettere in pratica le tue nuove conoscenze utilizzando gli esercizi alla fine di ogni capitolo.

1 – What is Computational Thinking?

Rispondere a questa domanda è in realtà piuttosto difficile. I sostenitori del pensiero computazionale (CT) fino a tempi molto recenti hanno dedicato molto tempo a discutere su come definirlo.

Ancora nel 2011, è stato organizzato un workshop in cui numerosi individui si sono riuniti per esplorare quale dovesse essere la natura del CT. Alcuni partecipanti a questo workshop hanno sostenuto la necessità di una definizione rigorosa e coerente (Committee for the Workshops on Computational Thinking, 2011). Al contrario, altri hanno sostenuto che cercare di definire rigorosamente il CT è inutile (Voogt et al., 2015).

Secondo questi ultimi, la comprensione del CT non dovrebbe avvenire attraverso una definizione nel senso usuale (in altre parole, creando un elenco di condizioni che qualcosa deve soddisfare prima di essere considerato una corrispondenza). Voogt et al. sostengono che definire il CT condivide difficoltà simili nel definire cosa sia un "gioco". (Ogni gioco deve mettere almeno un giocatore contro un altro? Ogni gioco ha il concetto di vincere? Ogni gioco dovrebbe includere un elemento di fortuna o casualità?) Come la nostra comprensione di un gioco, dicono, l'approccio alla definizione del CT dovrebbe essere più sfumato e considerato come una serie di somiglianze e relazioni che si intersecano e si sovrappongono.

Esistono altri motivi per rendere la definizione del CT un'impresa impegnativa. In primo luogo, il pensiero computazionale è fortemente legato all'informatica (CS), che di per sé può essere problematico definire in modo soddisfacente. Come l'CS, il CT include una serie di idee sia astratte che concrete. Entrambi condividono un'applicabilità universale e questa ampiezza, pur rendendolo potente, rende anche il CT difficile da definire in modo conciso.

Il CT è anche un'idea che è sia nuova che vecchia. È nuova nel senso che l'argomento è improvvisamente diventato un tema molto dibattuto nel 2006 dopo il discorso di Wing (Wing, 2006). Tuttavia, molte delle sue idee fondamentali sono già state discusse per diversi decenni e, lungo il percorso, le persone le hanno impacchettate in modi diversi. Ad esempio, già nel 1980, Seymour Papert del Massachusetts Institute of Technology ha introdotto una tecnica che ha chiamato "pensiero procedurale" (Papert, 1980). Condivideva molte idee con ciò che ora consideriamo CT. Usando il pensiero procedurale, Papert mirava a fornire agli studenti un metodo per risolvere i problemi usando i computer come strumenti. L'idea era che gli studenti avrebbero imparato a creare soluzioni algoritmiche che un computer avrebbe potuto eseguire; per questo ha usato il linguaggio di programmazione Logo. Gli scritti di Papert hanno ispirato molto nel CT, anche se il CT si è discostato da questa idea originale per alcuni aspetti.

Tuttavia, nei 10 anni successivi al discorso di Wing, sono stati tentati una serie di definizioni succinte. Una piccola parte di esse è riportata nella Tabella 1.1. Mentre accennano a idee simili, sembra esserci una certa diversità in ciò che queste persone dicono. Forse Voogt et al. avevano ragione e la nostra migliore speranza di comprendere il CT è costruire quei concetti sovrapposti e intersecanti. Fortunatamente, questo lavoro è già stato fatto da Cynthia Selby (Selby, 2013), quando ha setacciato la letteratura sul CT alla ricerca di concetti e li ha divisi in due categorie: concetti centrali per il CT e concetti che sono in qualche modo periferici e quindi dovrebbero essere esclusi da una definizione.

- "Il pensiero computazionale è il processo mentale coinvolto nella formulazione di un problema e nell'esprimere la sua/e soluzione/i in modo tale che un computer, umano o macchina, possa eseguirla efficacemente." (Wing, 2014)
- "L'attività mentale di astrarre problemi e formulare soluzioni che possono essere automatizzate." (Yadav et al., 2014)
- "Il processo di riconoscimento degli aspetti del calcolo nel mondo che ci circonda e di applicazione di strumenti e tecniche dell'Informatica per comprendere e ragionare sia sui sistemi e processi naturali che artificiali." (Furber, 2012)
- "Un orientamento mentale alla formulazione di problemi come conversioni da un input a un output e alla ricerca di algoritmi per eseguire le conversioni. Oggi il termine è stato ampliato per includere il pensiero con molti livelli di astrazioni, l'uso della matematica per sviluppare algoritmi e l'esame di quanto bene una soluzione si adatta a diverse dimensioni di problemi." (Denning, 2009)
- "[Insegnare CT significa insegnare] come pensare come un economista, un fisico, un artista e capire come usare il calcolo per risolvere i loro problemi, per creare e per scoprire nuove domande che possono essere esplorate con profitto." (Hemmendinger, 2010)

In questo libro, seguirò un approccio molto simile a quello di Selby. Alcuni concetti sono considerati "centrali" per il CT e sono trattati in dettaglio. Altri sono inclusi, ma trattati come periferici e coperti in modo molto meno dettagliato.

Questo libro considera i concetti centrali del CT come:

- pensiero logico;
- pensiero algoritmico;
- scomposizione;
- generalizzazione e riconoscimento di pattern;
- modellazione;
- astrazione;
- valutazione.

Altri concetti periferici saranno menzionati, ma non trattati come essenziali per l'argomento del CT. Essi includono:

- rappresentazione dei dati;
- pensiero critico;
- informatica;
- automazione;
- simulazione/visualizzazione.

Definirò i singoli concetti man mano che verranno introdotti nel corso del libro.

COME VIENE UTILIZZATO IL PENSIERO COMPUTAZIONALE?

Il CT può essere applicato da chiunque stia tentando di risolvere un problema e voglia che un computer giochi un ruolo nella soluzione. Per darci un'idea di come viene utilizzato il CT, non solo nell'informatica ma in una serie di aree tematiche, possiamo esaminare esempi (tratti da Barr e Stephenson (2011)) dei concetti fondamentali del pensiero computazionale appena elencati.

Ad esempio, cosa potrebbe significare il pensiero algoritmico in diverse situazioni? Per un informatico, significa lo studio degli algoritmi e la loro applicazione a diversi problemi. Per un matematico, potrebbe significare eseguire una divisione lunga, una fattorizzazione o fare riporti in addizione o sottrazione. Uno scienziato potrebbe pensarci come al processo di esecuzione di una procedura sperimentale.

Allo stesso modo, l'astrazione ha applicazioni al di là della visione dell'informatico. Quando un linguista usa similitudini e metafore, o scrive una storia con ramificazioni, sta usando l'astrazione, così come lo sono gli scienziati sociali che riassumono i fatti e li usano per trarre conclusioni. Quando uno scienziato costruisce un modello o un matematico usa l'algebra, anche loro hanno introdotto l'astrazione nel loro lavoro.

I seguenti sono esempi tratti da altre fonti che discutono esempi specifici di CT in azione.

Esempio: Organizzazione di una cerimonia di laurea

Il preside Randy Bryant stava riflettendo su come rendere più veloce la cerimonia di consegna dei diplomi. Posizionando attentamente le persone, ha progettato un efficiente sistema in modo che, dopo la lettura del nome e dei riconoscimenti di ciascun laureato da parte del vicedirettore Mark Stehlik, ogni persona potesse ricevere il proprio diploma, poi stringere la mano o ricevere un abbraccio da Mark e poi farsi scattare una foto. Questo sistema ha permesso un flusso costante di studenti sul palco (anche se si è verificato un intoppo quando il cappello del laureato cadeva mentre riceveva l'abbraccio da Mark). (Wing, 2011)

Esempio: Prevedere i cambiamenti climatici

Prevedere i cambiamenti climatici globali è possibile solo grazie a modelli computerizzati avanzati. Secondo il Met Office del Regno Unito, "L'unico modo per prevedere il tempo di giorno in giorno e i cambiamenti del clima su scale temporali più lunghe è utilizzare modelli computerizzati". (Furber, 2012)

Esempio: Ordinare classifiche musicali

Mi sono presentato a un concerto di big band e il capobanda ha distribuito libri con forse 200 partiture non ordinate e una scaletta con circa 40 titoli che dovevamo estrarre e mettere in ordine, pronti per suonare. Tutti gli altri hanno iniziato a cercare nella pila, estraendo le partiture una alla volta. Ho deciso di ordinare le 200 partiture alfabeticamente O(N log N) e poi estrarre le partiture O(M log(N)). Stavo ancora ordinando quando altri membri della band erano a metà delle loro partiture e ho iniziato a ricevere sguardi divertiti, ma alla fine ho finito per primo. Questo è pensiero computazionale. (Roger Dannenberg in Wing, 2011)

Esempio: Assistere polizia, avvocati e giudici

Il Pensiero Computazionale ha una lunga tradizione nell'influenzare il diritto, soprattutto nel sogno di fornire una serie di regole logiche che possano automatizzare il processo di raggiungimento di un verdetto, [sostenendo] il suo desiderio di minimizzare la discrezionalità umana e massimizzare la prevedibilità del risultato... i sistemi di ragionamento giuridico si stanno facendo strada laddove cercano semplicemente di assistere coloro che prendono decisioni legali. Ad esempio, i ricercatori del Joseph Bell Centre hanno costruito un sistema che costruisce uno spazio di ipotesi per spiegare le prove sulla scena di un crimine. Questo è stato usato per ricordare ai detective ipotesi che altrimenti avrebbero potuto trascurare. (Bundy, 2007)

DISCLAIMERS (Avvertenze)

Questa sezione discute i comuni malintesi sul CT, nonché le sue attuali carenze.

Cosa non è il pensiero computazionale?

È inevitabile che ci sia una certa confusione su cosa sia e cosa non sia il CT. Innanzitutto, è strettamente legato ad altre materie, come l'informatica e la programmazione, e distinguere tra loro può presentare difficoltà. Inoltre, la sua definizione è ampia ed è stata oggetto di dibattito per diversi anni. Questa sezione discuterà qualcosa che alcune persone potrebbero essere state portate a credere sul CT che non è necessariamente il caso. Nel processo, verranno rivelati maggiori dettagli sulla natura del CT.

Innanzitutto, insegnare il pensiero computazionale non è la stessa cosa che insegnare informatica. L'obiettivo principale dell'insegnamento di quest'ultima è educare gli studenti allo studio e all'applicazione dei principi del calcolo matematico. Forse per contrastare questa impressione, alcuni, tra cui Jeannette Wing, hanno detto che il pensiero computazionale è "pensare come un informatico". Tuttavia, altri hanno criticato questa frase (ad esempio, Denning, 2009; Hemmendinger, 2010) perché lascia inalterata una forte associazione con l'informatica, il che corre il rischio che le persone lo vedano non come un'abilità quotidiana, ma semplicemente come un riconfezionamento dell'CS.

Insegnare la programmazione (un sottocampo dell'CS) viene fatto principalmente per educare gli studenti su come scrivere al meglio i programmi e si concentra sulla produzione di software di alta qualità. Il CT, pur condividendo alcuni aspetti con queste altre materie, è meglio descritto come un approccio alla risoluzione dei problemi. Ciò che lo distingue da altri approcci alla risoluzione dei problemi è che il CT presuppone che un computer eseguirà la soluzione finale.

Il CT insegna un approccio alla risoluzione dei problemi in cui l'obiettivo finale è fornire una soluzione la cui forma significa che è pronta per essere programmata in un computer.

Il CT prende un sottoinsieme relativamente piccolo di concetti - che peraltro sono importanti per l'CS - e li usa per costruire un approccio alla risoluzione dei problemi ampiamente applicabile.

Queste distinzioni sono importanti. Insegnare competenze informatiche come parte obbligatoria del programma scolastico è già stato tentato in molti luoghi in tutto il mondo. Sebbene questi sforzi siano nobili e validi, le prove non sono conclusive sul fatto che abbiano successo o meno nel loro obiettivo di insegnare abilità trasferibili (Voogt et al., 2015); vale a dire, abilità che gli studenti possono intuitivamente portare con sé in vari ambiti. Il CT tenta di ottenere risultati migliori in questo senso. Distillando alcuni degli aspetti fondamentali dell'CS che sono rilevanti in molti altri campi, il CT può essere un corso nel programma scolastico di base che è rilevante per tutti. Nel curriculum di informatica del Regno Unito, ad esempio, il CT gioca già un ruolo importante (Department for Education, 2013). Può anche far parte di un corso di CS o di programmazione che si concentra esclusivamente sulla risoluzione di problemi basata sul computer.

Nel sottolineare le differenze tra informatica e pensiero computazionale, vale la pena sottolineare che i concetti di CT non sono affatto esclusivi dell'CS. Come sottolinea David Hemmendinger (2010), "Costruire modelli, trovare e correggere errori, disegnare diagrammi, analizzare - queste sono tutte parti di attività scientifiche (e di altro tipo)". Gli scienziati naturali, ad esempio, sostengono da tempo che il calcolo è una "terza gamba" del processo scientifico (insieme alla teoria

e alla sperimentazione) e che il pensiero computazionale è essenziale per il loro lavoro (Denning, 2009).

[L']obiettivo ultimo non dovrebbe essere insegnare a tutti a pensare come un informatico, ma piuttosto insegnare loro ad applicare questi elementi comuni per risolvere problemi e scoprire nuove domande che possono essere esplorate all'interno e attraverso tutte le discipline.

Carenze attuali

Molto presto procederemo a considerare i dettagli del pensiero computazionale. Prima che ciò accada, tuttavia, un'onesta discussione sull'argomento nella sua forma attuale dovrebbe affrontare eventuali carenze. Tieni presente che queste potrebbero scomparire con la maturazione del CT.

Maturità

Sebbene abbia influenze e predecessori che risalgono a diversi decenni fa, il CT come concetto formale è ancora relativamente immaturo. I workshop per esplorare il significato del CT sono stati organizzati solo cinque anni prima della stesura di questo libro. Solo tre anni fa, Cynthia Selby ha scritto un importante articolo proponendo una definizione di CT (Selby, 2013), perché le definizioni a quel tempo erano ancora vaghe. La maggior parte delle altre discipline ha da tempo smesso di organizzare incontri per decidere cosa fanno esattamente. Tuttavia, ora sta emergendo un consenso su cosa includere in una definizione, in gran parte in linea con la proposta di Selby di CT come "un approccio mirato alla risoluzione dei problemi, che incorpora processi di pensiero che utilizzano astrazione, scomposizione, algoritmi, valutazione e generalizzazioni" (2013). ("Cos'è il pensiero computazionale?" a p. 7.)

Efficacia

Un'ulteriore conseguenza della giovinezza del CT è la relativa scarsità di prove su quanto sia efficace l'insegnamento del CT. Finora non è stato insegnato ampiamente per un periodo di tempo prolungato. Tuttavia, il corpus di prove crescerà man mano che verranno condotti studi sulla sua efficacia. Rapporti di esperienza e testimonianze forniscono una forma di prova più immediata.

Imperialismo percepito

Altri commentatori hanno avvertito che i sostenitori del CT rischiano di essere imperialisti e scoraggianti. Le loro affermazioni come "dovresti pensare come un informatico" o "il pensiero computazionale è X" (dove X è qualcosa che approvano) sembrano territoriali. Come ha detto David Hemmendinger (2010), i sostenitori dovrebbero fare attenzione che il CT non prenda l'abitudine di dire: "Se è un buon modo di pensare, allora è nostro". Va notato che molte pratiche nel CT non sono originali dell'argomento. Infatti, non sono tutte originali dell'informatica; molte di esse sono state utilizzate in altri campi e nella vita di tutti i giorni per molto tempo. Non diventano semplicemente computazionali per il fatto di trovare applicazione nell'informatica.

Tuttavia, ciò mitiga in qualche modo l'immaturità del CT quando ti rendi conto che le sue parti costituenti (come la logica, gli algoritmi e la scomposizione) sono individualmente provate, testate e mature.